

Trabajo Práctico

Modelado y Optimización

Profesor:

Marcelo Mydlarz

Primer semestre 2026

Universidad Nacional de General Sarmiento

Integrantes:

Aragón, Evelin

Faccini, Gonzalo

Curcio, Matías

Torres, Pablo

Contents

0	Introducción	3
(a)	Descripción del Problema	3
(b)	Desafíos Operativos	3
(c)	Objetivo General	3
1	Estrategia 1: Salud (Modelo MILP Compacto)	4
(a)	Descripción de la Estrategia	4
(b)	Modelo Matemático	4
i	Descripción del Problema Específico	4
ii	Conjuntos, Parámetros y Variables de Decisión	4
iii	Formulación Algebraica Completa	5
iv	Explicación de la Función Objetivo y Restricciones	5
(c)	Pseudocódigo y Detalles de Implementación	6
i	Detalles de implementación	6
2	Estrategia 2: SaludCG (Generación de Columnas)	8
(a)	Descripción de la Estrategia	8
(b)	Modelo Matemático	8
i	Descripción del Problema Específico	8
ii	Conjuntos, Parámetros y Variables de Decisión	8
iii	Formulación Algebraica Completa	9
iv	Explicación de la Función Objetivo y Restricciones	10
(c)	Pseudocódigo y Detalles de Implementación	11
i	Detalles de implementación	11
3	Estrategia 3: SaludChallenger (Branch & Price)	13
(a)	Descripción de la Estrategia	13
(b)	Modelo Matemático	14
i	Descripción del Problema Específico	14
ii	Conjuntos, Parámetros y Variables de Decisión	14
iii	Formulación Algebraica Completa	15
iv	Explicación de la Función Objetivo y Restricciones	16
(c)	Pseudocódigo y Detalles de Implementación	16
i	Detalles de implementación	20
4	Evaluación de las estrategias	21
(a)	Configuración de Evaluación	21
(b)	Tabla Comparativa	21
(c)	Abreviaciones	22
5	Conclusiones	24
(a)	Hallazgos Principales	24
(b)	Comportamientos Sorprendentes	24
(c)	Dificultades y Conocimientos Adquiridos	25
6	Apéndice: módulo de validación SaludTest	27

0 Introducción

El trabajo práctico se enfoca en resolver un problema fundamental de logística de salud integral: optimizar el traslado de pacientes desde sus domicilios hacia un centro médico de alta complejidad, en contextos donde los recursos (vehículos) son limitados y las restricciones operativas son complejas.

(a) Descripción del Problema

Una empresa de logística médica opera una flota heterogénea de vehículos (combis) basadas en un centro médico central. Cada día, debe atender solicitudes de traslado de pacientes desde sus hogares al centro. Debido a la naturaleza médica de los traslados y a limitaciones de recursos, la empresa no está obligada a recoger a todos los pacientes, sino que debe seleccionar estratégicamente cuáles atender para maximizar el beneficio social y operativo del sistema.

Cada paciente presenta características específicas:

- **Ubicación geográfica:** coordenadas (x_i, y_i) en el plano
- **Ventana horaria:** intervalo $[ih_{\text{inicio}}, ih_{\text{fin}}]$ dentro del cual debe ser recogido
- **Categoría médica:** clasificación según protocolo médico (p.ej., inmunodeprimido, infeccioso, movilidad reducida)
- **Beneficio:** valor social/económico de atender al paciente

La flota de vehículos también es heterogénea:

- **Capacidad de asientos:** cantidad máxima de pacientes por vehículo
- **Costo operativo:** costo de poner en servicio el vehículo por día
- **Disponibilidad:** cantidad de unidades de cada tipo

(b) Desafíos Operativos

- **Incompatibilidades médicas:** Ciertas categorías de pacientes no pueden compartir vehículo por razones de bioseguridad o protocolo médico.
- **Ventanas de tiempo:** Cada paciente tiene una ventana horaria estricta. No se permite recoger antes de ih_{inicio} ni después de ih_{fin} .
- **Selección estratégica:** No todos los pacientes pueden ser atendidos; debe maximizarse el beneficio neto (beneficio de pacientes atendidos menos costo operativo de vehículos).
- **Optimización de rutas:** Diseñar rutas eficientes que respeten capacidad, tiempos y compatibilidades.
- **Heterogeneidad de recursos:** Diferentes tipos de vehículos con distintas características implican complejidad combinatoria.

(c) Objetivo General

Desarrollar e implementar tres estrategias de resolución con complejidad creciente:

1. **Salud (Modelo MILP Compacto):** Formulación algebraica directa utilizando programación lineal entera mixta.
2. **SaludCG (Generación de Columnas):** Descomposición del problema mediante generación de columnas para mejorar escalabilidad.
3. **SaludChallenger (Branch & Price):** Mejoras avanzadas basadas en la literatura (descomposición, eliminación de columnas, inicialización eficiente).

Todas las estrategias deben incluir un módulo de validación (**SaludTest**) que verifica factibilidad sin usar programación lineal.

1 Estrategia 1: Salud (Modelo MILP Compacto)

(a) Descripción de la Estrategia

La estrategia Salud resuelve el problema mediante un modelo de **Programación Lineal Entera Mixta (MILP) compacto**. Este enfoque formula el problema completo en un único modelo algebraico que incluye todas las restricciones operativas, y lo resuelve directamente utilizando un solver de optimización (en este caso, PySCIPOpt sobre el solver SCIP).

Ventajas:

- Garantiza optimalidad (o cotas de optimalidad) dentro del tiempo permitido
- Implementación directa y comprensible
- Adecuada para instancias pequeñas a medianas

Limitaciones:

- Escalabilidad limitada para instancias grandes
- Tiempo de resolución puede crecer exponencialmente con el tamaño
- Espacio de soluciones potencialmente muy grande

(b) Modelo Matemático

i Descripción del Problema Específico

Diseñar simultáneamente el recorrido de todos los vehículos de la flota: qué pacientes se atienden, a qué combi se asigna cada uno y en qué orden los visita, respetando capacidad, ventanas horarias e incompatibilidades, de modo de maximizar el beneficio neto. A diferencia de las Estrategias 2 y 3, aquí todo se resuelve en un único modelo: no hay descomposición ni generación dinámica de variables.

ii Conjuntos, Parámetros y Variables de Decisión

Conjuntos:

- P : Conjunto de pacientes solicitantes, $P = \{1, 2, \dots, m\}$
- $V = \{0\} \cup P$: Conjunto de nodos (centro médico + pacientes)
- K : Conjunto de combis individuales disponibles
- T : Conjunto de tipos de combis (Combi_Chica, Combi_Mediana, etc.)
- $\mathcal{C}$: Conjunto de categorías médicas
- $\mathcal{I} \subseteq \mathcal{C} \times \mathcal{C}$: Conjunto de pares de categorías incompatibles

Parámetros:

- $\text{beneficio}[p]$: Beneficio de atender al paciente p
- $\text{costo}[t]$: Costo operativo del tipo de combi t ; $\text{tipo}[k]$: tipo al que pertenece la combi k
- $\text{capacidad}[k]$: Número máximo de asientos en la combi k
- $(x_p, y_p), (x_0, y_0)$: Coordenadas del paciente p y del centro médico
- $d[i, j] = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$: Distancia euclídea entre los nodos i y j
- $[\text{ih}_{\text{inicio}}[p], \text{ih}_{\text{fin}}[p]]$: Ventana de tiempo del paciente p ; $\text{categoria}[p]$: su categoría médica
- $M = 10000$: Constante Big-M para la linealización de las restricciones lógicas

Variables de decisión:

- $x[i, j, k] \in \{0, 1\}$: la combi k viaja directamente del nodo i al nodo j
- $z[p, k] \in \{0, 1\}$: el paciente p es atendido por la combi k
- $a[p] \in \{0, 1\}$: el paciente p es atendido por alguna combi
- $u[k] \in \{0, 1\}$: la combi k es utilizada (realiza algún viaje)
- $T[i, k] \geq 0$: tiempo de llegada (o inicio de servicio) al nodo i con la combi k

iii Formulación Algebraica Completa

Con $\mathcal{I}_{\text{pacientes}} = \{(p, q) : (\text{categoria}[p], \text{categoria}[q]) \in \mathcal{I}\}$:

$$\max \quad Z = \sum_{p \in P} \text{beneficio}[p] \cdot a[p] - \sum_{k \in K} \text{costo}[\text{tipo}[k]] \cdot u[k] \quad (1)$$

$$\text{s.a.} \quad \sum_{k \in K} z[p, k] \leq 1 \quad \forall p \in P \quad (2)$$

$$a[p] = \sum_{k \in K} z[p, k] \quad \forall p \in P \quad (3)$$

$$\sum_{j \in V, j \neq i} x[i, j, k] = \sum_{j \in V, j \neq i} x[j, i, k] \quad \forall i \in V, k \in K \quad (4)$$

$$\sum_{j \in P} x[0, j, k] = u[k] \quad \forall k \in K \quad (5)$$

$$\sum_{i \in P} x[i, 0, k] = u[k] \quad \forall k \in K \quad (6)$$

$$\sum_{p \in P} z[p, k] \leq \text{capacidad}[k] \cdot u[k] \quad \forall k \in K \quad (7)$$

$$T[0, k] = 0 \quad \forall k \in K \quad (8)$$

$$T[i, k] \geq T[j, k] + d[j, i] - M(1 - x[j, i, k]) \quad \forall j \in V, i \in P, k \in K, i \neq j \quad (9)$$

$$T[p, k] \geq ih_{\text{inicio}}[p] - M(1 - z[p, k]) \quad \forall p \in P, k \in K \quad (10)$$

$$T[p, k] \leq ih_{\text{fin}}[p] + M(1 - z[p, k]) \quad \forall p \in P, k \in K \quad (11)$$

$$z[p, k] + z[q, k] \leq 1 \quad \forall k \in K, (p, q) \in \mathcal{I}_{\text{pacientes}} \quad (12)$$

$$z[p, k] = \sum_{i \in V, i \neq p} x[i, p, k] \quad \forall k \in K, p \in P \quad (13)$$

iv Explicación de la Función Objetivo y Restricciones

Función objetivo (1). Maximiza el *beneficio neto*: el primer término suma el beneficio de los pacientes atendidos y el segundo resta el costo de operación de las combis efectivamente utilizadas. El modelo busca el equilibrio entre atender más pacientes y no poner en servicio vehículos que no se paguen a sí mismos.

Cobertura (2), (3). La primera impide que un paciente sea recogido por más de una combi; la desigualdad (en vez de igualdad) es la que permite dejarlo sin atender, tal como admite el enunciado. La segunda define $a[p]$ como indicador de que p fue atendido por alguna combi, y es la que conecta la cobertura con el objetivo.

Conservación de flujo (4). Todo lo que “entra” a un nodo debe “salir” de él, de modo que los arcos elegidos por cada combi formen caminos y no tramos sueltos.

Por sí sola esta restricción *no* elimina los subtours: un ciclo entre pacientes desconectado del centro también conserva el flujo. Quien los elimina es la propagación temporal (9), que al recorrer un ciclo obligaría a $\sum d > 0$ a ser ≤ 0 . La única excepción es un ciclo de longitud nula (pacientes con coordenadas idénticas); ninguna de las instancias evaluadas presenta domicilios repetidos, pero es una limitación conocida de esta formulación, que la Estrategia 3 resuelve agregando un orden de tipo MTZ (Ec. (38)).

Operación del centro (5), (6). Si la combi k se utiliza ($u[k] = 1$), sale exactamente una vez del centro y regresa exactamente una vez a él; si no se utiliza, no puede usar ningún arco incidente al centro. Esto codifica la condición del enunciado de que cada combi realiza a lo sumo un viaje.

Capacidad (7). La cantidad de pacientes asignados a la combi k no puede superar sus asientos. Al multiplicar por $u[k]$, la restricción también fuerza $z[p, k] = 0$ para toda combi no utilizada.

Ventanas de tiempo (8)–(11). El viaje comienza en $t = 0$ en el centro. La propagación (9) arrastra el tiempo a lo largo de los arcos usados; al ser una desigualdad, la combi puede *esperar* en la puerta hasta ih_{inicio} , tal como exige el enunciado. Las dos últimas obligan a que cada paciente atendido sea recogido dentro de su ventana; el término $M(1 - z[p, k])$ las desactiva para las combis que no lo atienden.

Incompatibilidades (12). Dos pacientes de categorías incompatibles no pueden compartir vehículo en ningún momento del trayecto. Como cada combi realiza un único viaje y nadie desciende antes del centro, basta prohibir que ambos estén asignados a la misma combi.

Vínculo entre visita y atención (13). Cierra las dos implicaciones: si la combi entra al nodo de un paciente entonces lo recoge, y si lo recoge entonces entró a su nodo. Usar igualdad (en lugar de $\leq$) ajusta el espacio factible del LP y evita soluciones degeneradas en las que una combi atraviesa un nodo sin levantar al paciente. La sumatoria recorre todo V (no solo P) para contabilizar también los arcos directos centro $\rightarrow$ paciente.

(c) Pseudocódigo y Detalles de Implementación

Al tratarse de un modelo compacto, no hay un algoritmo de búsqueda propio: la exploración la realiza íntegramente SCIP. Lo que sí merece describirse es el flujo que rodea al solver (Algoritmo 1.1), en particular el reparto del umbral de tiempo y la *reconstrucción de las rutas* a partir de las variables x , que no es inmediata.

Algoritmo 1.1 SALUD — modelo compacto MILP

Entrada: instancia (pacientes P , centro, flota, incompatibilidades), umbral de tiempo

Salida: archivo de salida con la mejor solución entera hallada

- 1: $inicio \leftarrow$ ahora; leer la instancia y calcular la matriz de distancias
 - 2: $K \leftarrow$ una copia individual por cada unidad disponible de cada tipo de combi
 - 3: construir el MILP con las variables x, z, a, u, T y las restricciones (2)–(13)
 - 4: reportar |variables| y |restricciones| ▷ del modelo *inicial*, antes del presolve
 - 5: $t_{\text{solver}} \leftarrow$ umbral $-(\text{ahora} - inicio) - \text{reserva}$ ▷ descuenta lectura y armado
 - 6: fijar `limits/time` $\leftarrow t_{\text{solver}}$ y optimizar
 - 7: reportar la cota dual que informa el solver al detenerse
 - 8: **para cada** combi k con $u_k > 0,5$ **hacer** ▷ reconstrucción de la ruta
 - 9: $S \leftarrow \{p : z_{pk} > 0,5\}$ ▷ pacientes que recoge
 - 10: $A \leftarrow \{i \mapsto j : x_{ijk} > 0,5\}$; recorrer A desde el centro hasta volver a él
 - 11: **si** el recorrido no cubre exactamente S **entonces** ordenar S por id ▷ resguardo
 - 12: **fin si**
 - 13: emitir la línea $\langle \text{tipo}(k) \rangle$: $[0 \rightarrow \dots \rightarrow 0]$
 - 14: **fin para**
 - 15: escribir Z , las rutas y los pacientes con $a_p < 0,5$ como no atendidos
-

i Detalles de implementación

- **Una variable por unidad de combi.** La flota se “desdobla”: si hay 5 `Combi_Chica`, el conjunto K contiene 5 combis individuales, cada una con su propio juego de variables. Esto hace el modelo directo de escribir, pero introduce *simetría* — las unidades de un mismo tipo son intercambiables —, que es una de las razones por las que el compacto escala mal (véase la Sección 5).

- **Tamaño reportado.** Las métricas de la Sección 4 se emiten *antes* de llamar al solver: una vez ejecutado, SCIP informa el modelo ya reducido por el presolve, que puede ser menos de la mitad del original y no es lo que pide el enunciado.
- **Expiración segura del umbral.** El presupuesto se cuenta desde el inicio de la función, de modo que la lectura y el armado del modelo se descuentan del tiempo que recibe el solver; además se reserva un margen de $\min(5, \max(1, 0,02 \cdot \text{umbral}))$ segundos para extraer la solución y escribir la salida. Si el umbral se agota, SCIP devuelve la mejor solución entera encontrada hasta ese momento.
- **Big-M.** Se usa $M = 10^4$, holgado frente a las ventanas horarias de las instancias. Un M innecesariamente grande debilita la relajación lineal, y ése es el origen de las cotas duales flojas que se comentan en las conclusiones.
- **Reconstrucción de rutas.** El orden de visita no es una variable del modelo: se recupera encadenando los arcos $x_{ijk} = 1$ desde el centro. El recorrido corta si detecta un nodo repetido, y si el camino resultante no coincide con el conjunto de pacientes asignados se cae a un orden por id, de modo que la salida siempre queda bien formada.

2 Estrategia 2: SaludCG (Generación de Columnas)

(a) Descripción de la Estrategia

La estrategia SaludCG implementa un enfoque de **Generación de Columnas (Column Generation, CG)** para resolver el problema de ruteo de vehículos de manera escalable y estructurada. A diferencia de la Estrategia 1, que evalúa todas las combinaciones topológicas simultáneamente en un modelo compacto, este método descompone el problema original en dos componentes matemáticos que dialogan entre sí:

- **Problema Maestro:** Evalúa un *pool* (subconjunto) restringido de rutas previamente construidas y selecciona la combinación óptima para maximizar la rentabilidad, determinando a su vez el valor marginal de cada recurso.
- **Subproblema de Pricing:** Es un modelo MILP independiente por cada tipo de vehículo. Utiliza los precios sombra del Maestro para explorar el espacio de soluciones y descubrir nuevas rutas que, de agregarse al pool, mejorarían el beneficio global.

El proceso ejecuta un bucle continuo entre ambos modelos. Finaliza únicamente cuando los subproblemas de Pricing demuestran matemáticamente que ya no existe ninguna ruta capaz de mejorar la solución actual, o cuando se activa un resguardo de tiempo límite.

(b) Modelo Matemático

i Descripción del Problema Específico

Problema Maestro Relajado Decidir *qué rutas ejecutar* para maximizar el beneficio neto, dado un conjunto de rutas ya construidas. El Problema Maestro abstrae la topología del ruteo y se concentra puramente en la selección de conjuntos: toda la factibilidad del recorrido (capacidad, ventanas horarias, incompatibilidades) ya está garantizada en la construcción de cada columna. Sea Ω el conjunto dinámico de rutas generadas hasta el momento, y $\Omega_k \subseteq \Omega$ las rutas pertenecientes al tipo de vehículo k .

Subproblema de Pricing Por cada iteración y tipo de combi k , hallar la ruta factible de *máxima ganancia reducida*: aquella que, de incorporarse al pool, más mejoraría el valor del maestro. Es un problema de ruteo de un solo vehículo con selección de pacientes y ventanas de tiempo (VRPTW con *prizes*), que concentra toda la complejidad topológica que el maestro delega.

ii Conjuntos, Parámetros y Variables de Decisión

Problema Maestro Relajado Conjuntos y Parámetros:

- P : Conjunto de todos los pacientes solicitantes.
- K : Conjunto de los tipos de vehículos disponibles en la flota.
- Ω : Conjunto dinámico de rutas generadas hasta el momento en el *pool*.
- $\Omega_k \subseteq \Omega$: Subconjunto de rutas factibles pertenecientes al tipo de vehículo $k \in K$.
- rentabilidad_r : Beneficio neto de la ruta $r \in \Omega$ (suma de los beneficios de los pacientes atendidos menos el costo operativo de la combi).
- cant_disponible_k : Stock máximo o cantidad de unidades físicas disponibles del vehículo tipo k .
- $p \in r$: Condición lógica que indica que el paciente p es visitado dentro de la secuencia de la ruta r .

Variable de Decisión:

- $y_r \in [0, 1]$: Variable continua que representa la fracción de utilización de la ruta $r \in \Omega$.

Subproblema de Pricing Conjuntos y Parámetros del Subproblema:

- P : Conjunto de pacientes solicitantes.
- $V = \{0\} \cup P$: Conjunto de nodos (centro médico 0 y pacientes P).
- beneficio_p : Beneficio económico/social de atender al paciente p .
- costo_operacion_k : Costo fijo de poner en servicio un vehículo del tipo k .
- asientos_k : Capacidad máxima de asientos del vehículo tipo k .
- $d_{i,j}$: Distancia euclidiana entre el nodo i y el nodo j .
- $[ih_{\text{inicio}}[p], ih_{\text{fin}}[p]]$: Ventana de tiempo estricta para la atención del paciente p .
- $\mathcal{I}$: Conjunto de pares de categorías de pacientes incompatibles por bioseguridad.
- π_p : Precios sombra (dual) del maestro asociados a la cobertura de cada paciente p .
- μ_k : Precio sombra (dual) asociado a la disponibilidad del vehículo tipo k .
- M : Constante grande (Big-M) para la linealización lógica de subtours y ventanas de tiempo.

Variables de Decisión del Subproblema:

- $x_{i,j} \in \{0, 1\}$: Variable binaria, igual a 1 si el vehículo viaja directamente del nodo i al nodo j .
- $z_p \in \{0, 1\}$: Variable binaria, igual a 1 si el paciente p es seleccionado y atendido en la ruta.
- $T_i \geq 0$: Variable continua que indica el tiempo de llegada o inicio de servicio en el nodo i .

iii Formulación Algebraica Completa

Problema Maestro Relajado

$$\max \quad Z = \sum_{r \in \Omega} \text{rentabilidad}_r \cdot y_r \quad (14)$$

$$\text{s.a.} \quad \sum_{r \in \Omega: p \in r} y_r \leq 1 \quad \forall p \in P \quad (\text{Dual: } \pi_p \geq 0) \quad (15)$$

$$\sum_{r \in \Omega_k} y_r \leq \text{cant_disponible}_k \quad \forall k \in K \quad (\text{Dual: } \mu_k \geq 0) \quad (16)$$

Subproblema de Pricing

$$\max \quad \bar{c} = \sum_{p \in P} (\text{beneficio}_p - \pi_p) \cdot z_p - \text{costo_operacion}_k - \mu_k \quad (17)$$

$$\text{s.a.} \quad \sum_{p \in P} z_p \leq \text{asientos}_k \quad (18)$$

$$\sum_{i \in V, i \neq p} x_{i,p} = z_p \quad \forall p \in P \quad (19)$$

$$\sum_{j \in V, j \neq p} x_{p,j} = z_p \quad \forall p \in P \quad (20)$$

$$\sum_{j \in P} x_{0,j} \leq 1 \quad (21)$$

$$\sum_{i \in P} x_{i,0} = \sum_{j \in P} x_{0,j} \quad (22)$$

$$\sum_{p \in P} z_p \geq \sum_{j \in P} x_{0,j} \quad (23)$$

$$T_j \geq T_i + d_{i,j} - M(1 - x_{i,j}) \quad \forall i, j \in V, i \neq j, j \neq 0 \quad (24)$$

$$T_p \geq ih_{\text{inicio}}[p] \cdot z_p \quad \forall p \in P \quad (25)$$

$$T_p \leq ih_{\text{fin}}[p] \cdot z_p + M(1 - z_p) \quad \forall p \in P \quad (26)$$

$$z_{p_1} + z_{p_2} \leq 1 \quad \forall (p_1, p_2) \in \mathcal{I} \quad (27)$$

iv Explicación de la Función Objetivo y Restricciones

Problema Maestro Relajado El objetivo (14) suma la rentabilidad neta de las rutas seleccionadas; coincide con el beneficio neto del enunciado porque cada columna ya descuenta el costo operativo de su combi. La restricción (15) garantiza que cada paciente se atienda a lo sumo una vez (la desigualdad permite dejarlo sin cubrir, tal como admite el enunciado). De aquí se extrae el dual π_p , que funciona como el “precio sombra” o costo de oportunidad que el sistema está pagando por ese paciente. La restricción (16) limita el uso de la flota al stock real, extrayendo el dual μ_k que penaliza el agotamiento de vehículos. En el maestro relajado y_r es continua; la integralidad se recupera al final resolviendo el maestro entero sobre el pool acumulado.

Subproblema de Pricing El objetivo (17) maximiza la ganancia reducida de la columna: cada paciente “vale” su beneficio menos el precio sombra π_p que el maestro ya está pagando por cubrirlo, y se descuentan las constantes costo_operacion_k y μ_k , que no dependen de las variables.

- **Capacidad ((18)):** Limita la cantidad máxima de pacientes que puede recoger la combi según sus asientos disponibles.
- **Flujo de nodos ((19), (20)):** Asegura que si un paciente es atendido ($z_p = 1$), obligatoriamente una arista entra y otra sale de su nodo; si no es atendido, el flujo es cero.
- **Operación del depósito ((21), (22), (23)):** Regula que la combi realice a lo sumo un viaje partiendo y regresando al centro médico, obligándola a visitar al menos un paciente si decide salir a operar.
- **Eliminación de subtours y tiempos ((24), (25), (26)):** Propagan el tiempo de recorrido a lo largo de las aristas y garantizan que la atención de cada paciente seleccionado ocurra estrictamente dentro de su ventana horaria de servicio.
- **Incompatibilidades médicas ((27)):** Evitan que la ruta agrupe a pacientes cuyas categorías pertenezcan al conjunto de restricciones de bioseguridad $\mathcal{I}$.

Si el solver encuentra una solución factible cuya ganancia reducida $\bar{c}$ es estrictamente mayor que 10^{-4} , la secuencia de nodos se traduce a un formato estructurado y se inyecta como una nueva columna r en el *pool* del Maestro.

(c) Pseudocódigo y Detalles de Implementación

El Algoritmo 2.1 resume el esquema completo: se parte de un pool inicial construido heurísticamente y se alterna maestro $\leftrightarrow$ pricing hasta que ningún tipo de combi aporte columnas atractivas, momento en el cual se resuelve el maestro entero sobre el pool acumulado para recuperar una solución factible.

Algoritmo 2.1 SALUDCG — generación de columnas

Entrada: instancia (pacientes P , centro, flota, incompatibilidades), umbral de tiempo

Salida: archivo de salida con la mejor solución entera hallada

```

1: leer la instancia y calcular la matriz de distancias; inicio  $\leftarrow$  ahora
2: construir un MILP de pricing por cada tipo de combi  $k$  ▷ se arma una sola vez
3:  $\Omega \leftarrow \text{RUTASINICIALES}()$  ▷ directas + golosas; ver más abajo
4: repetir
5:   si  $\text{ahora} - \text{inicio} > 0,8 \cdot \text{umbral}$  entonces salir del ciclo ▷ se reserva el 20% final para el
     maestro entero
6:   fin si
7:   resolver el LP maestro relajado sobre  $\Omega \Rightarrow$  duales  $\pi_p$  y  $\mu_k$ 
8:    $\text{nuevas} \leftarrow 0$ 
9:   para cada tipo de combi  $k$  hacer
10:    actualizar el objetivo del pricing de  $k$  con  $(\pi, \mu_k)$  y resolverlo ▷ límite de 10 s
11:    si  $\bar{c} > 10^{-4}$  y la ruta no está ya en  $\Omega$  entonces
12:      agregar la ruta a  $\Omega$ ;  $\text{nuevas} \leftarrow \text{nuevas} + 1$ 
13:    fin si
14:  fin para
15:  si  $\text{nuevas} = 0$  entonces salir del ciclo ▷ el LP maestro es óptimo
16:  fin si
17: fin repetir
18: resolver el maestro entero ( $y_r$  binaria) sobre  $\Omega$  con el tiempo restante
19: escribir  $Z$ , el itinerario de cada ruta elegida y los pacientes no atendidos

```

Dos detalles merecen comentario. Primero, el corte por tiempo es *doble*: el bucle de generación se detiene al 80% del umbral para que siempre quede presupuesto con el que resolver el maestro entero, que es el que produce la solución factible que se escribe. Segundo, los modelos de pricing se construyen una única vez y entre iteraciones solo se les actualiza la función objetivo con los duales nuevos (vía `freeTransform` de PySCIPOpt), siguiendo la recomendación del enunciado de modificar dinámicamente un modelo ya ejecutado en lugar de reconstruirlo desde cero.

i Detalles de implementación

Inicialización de rutas iniciales. Para garantizar una convergencia rápida y eficiente del algoritmo de Generación de Columnas, la estrategia no inicia con un pool de rutas vacío, sino que implementa una fase preliminar de inicialización inteligente en el módulo `utils_saludCG.py`. Esta etapa combina rutas directas y enfoques heurísticos golosos para poblar el conjunto Ω con opciones de alta calidad desde la primera iteración:

- **Rutas Directas (`generar_rutas_directas`):** Se construyen trayectos individuales de ida y vuelta desde el centro médico hacia cada paciente que cumpla de manera factible con las restricciones temporales iniciales, asegurando una cobertura básica individual por cada tipo de vehículo compatible.
- **Construcción Golosa Diversificada (`generar_rutas_golosas`):** Se generan rutas multi-paciente complejas mediante tres criterios de priorización independientes:
 1. *Priorizar Beneficio*: Selecciona iterativamente los pacientes más rentables que respeten la capacidad de la combi, las ventanas horarias y las normas de bioseguridad.
 2. *Priorizar Distancia*: Agrupa a los pacientes evaluando la cercanía geográfica respecto al centro médico o la posición actual del recorrido.
 3. *Priorizar Coeficiente ($Ratio\ Beneficio/Distancia$)*: Maximiza la eficiencia económica por unidad de distancia recorrida.
- **Filtrado de Unicidad (`filtrar_rutas_unicas`):** Todas las rutas candidatas generadas se procesan mediante una clave de control basada en el tipo de vehículo y la secuencia exacta del camino, eliminando duplicados para mantener la matriz del Problema Maestro limpia y sin redundancias.

3 Estrategia 3: SaludChallenger (Branch & Price)

(a) Descripción de la Estrategia

La estrategia **SaludChallenger** convierte la generación de columnas de la Estrategia 2 en un método *exacto* mediante **Branch & Price (B&P)**, la técnica estándar de la literatura para problemas de ruteo resueltos por generación de columnas. La motivación surge de dos limitaciones de SaludCG:

- **Su etapa final es heurística.** Cuando SaludCG deja de generar columnas, resuelve el maestro *entero* usando únicamente las rutas de su pool. Nada garantiza que la solución entera óptima se forme con esas rutas: puede necesitar columnas que la fase LP nunca tuvo motivo de generar.
- **No tiene mecanismo para cerrar el gap.** La cota dual de SaludCG corresponde a la relajación lineal del nodo raíz; si esa relajación es fraccionaria, el método no dispone de ninguna herramienta para acercarse a la cota al valor de la mejor solución entera.

Branch & Price ataca ambos problemas integrando la generación de columnas dentro de un árbol de *branch & bound*: la relajación lineal de **cada nodo** del árbol se resuelve por generación de columnas (con lo cual la cota de cada nodo es válida aunque el maestro tenga pocas columnas), y cuando la solución de un nodo es fraccionaria se **ramifica** usando reglas elegidas especialmente para que las decisiones puedan imponerse también dentro del subproblema de pricing. Con tiempo suficiente, el método encuentra el óptimo entero y además *certifica* su optimalidad.

Vista general del método

El algoritmo se organiza en seis pasos; los pseudocódigos correspondientes están en la Sección (c).

1. **Inicialización del pool de columnas** (Algoritmo 3.3): se generan todas las rutas *singleton* factibles (ida y vuelta a cada paciente alcanzable, por cada tipo de combi) más rutas multi-paciente construidas por una heurística golosa que respeta ventanas de tiempo, capacidad e incompatibilidades.
2. **Warm-start del incumbente:** antes de explorar el árbol se resuelve el maestro entero sobre el pool inicial. Su valor (o $Z = 0$, “no operar”, que siempre es factible) es el primer incumbente y habilita podas desde el primer nodo.
3. **Exploración best-first:** los nodos abiertos se guardan en una cola de prioridad ordenada por la cota LP de su padre; siempre se procesa primero el nodo más prometedor.
4. **Generación de columnas en el nodo** (Algoritmo 3.2): se resuelve la relajación lineal del maestro restringido a las columnas compatibles con las decisiones del nodo, iterando maestro $\leftrightarrow$ pricing hasta que ningún tipo de combi aporte columnas de ganancia reducida positiva. Durante estas iteraciones se *eliminan* las columnas que llevan K resoluciones consecutivas sin usarse.
5. **Podar o ramificación:** si la cota LP del nodo no supera al incumbente, el nodo se descarta; si la solución es entera, se actualiza el incumbente; si es fraccionaria, se crean dos hijos con la primera regla aplicable (Ryan–Foster sobre pares de pacientes, atención de un paciente, o cantidad de vehículos por tipo).
6. **Refuerzo final:** al agotarse el árbol o el tiempo, se resuelve una última vez el maestro entero sobre *todo* el pool acumulado, que puede mejorar el incumbente combinando rutas generadas en distintos nodos.

Relación con las mejoras sugeridas en el enunciado

- *Generar las columnas iniciales de manera más eficiente:* singletons factibles más heurística golosa (Algoritmo 3.3); el pool inicial ya contiene rutas de buena calidad, y el warm-start las convierte de inmediato en una solución factible.

- *Eliminar las columnas no usadas en las últimas iteraciones:* contador *sin_uso* por columna con umbral $K = 25$. Las columnas de la mejor solución conocida están protegidas y, si el pricing vuelve a generar una columna eliminada, ésta se reactiva; así la limpieza no invalida las cotas (detalles en la Sección i).
- *Mejoras basadas en la literatura:* el esquema completo de Branch & Price con ramificación de Ryan–Foster.

Ventajas:

- Método exacto: si el árbol se agota dentro del umbral, la solución queda *certificada* como óptima (a diferencia de la etapa final heurística de SaludCG)
- Cotas duales válidas en todo momento, lo que permite reportar el gap de optimalidad al detenerse por tiempo
- El maestro de cada nodo trabaja con pocas variables (solo las rutas generadas y compatibles con el nodo)

Limitaciones:

- El subproblema de pricing sigue siendo un MILP de ruteo (NP-difícil); es el cuello de botella en instancias grandes
- El árbol puede crecer en instancias con mucha simetría (columnas intercambiables entre combis idénticas)

(b) Modelo Matemático

La estrategia utiliza dos modelos que dialogan entre sí: el **problema maestro**, que selecciona rutas, y el **subproblema de pricing**, que construye rutas nuevas. Se describen a continuación, junto con las reglas de ramificación que los conectan con el árbol de búsqueda.

i Descripción del Problema Específico

Problema Maestro (formulación por rutas) Decidir *qué rutas ejecutar* para maximizar el beneficio neto, suponiendo conocido el conjunto de todas las rutas factibles. Una *ruta* es el recorrido de una combi que parte del centro, recoge un subconjunto de pacientes en cierto orden y regresa al centro, respetando capacidad, ventanas de tiempo e incompatibilidades. Esta formulación traslada toda la complejidad del ruteo a la definición de las columnas: al maestro solo le queda combinarlas.

Subproblema de Pricing Dados los precios sombra del maestro restringido, encontrar —para cada tipo de combi t — la ruta factible de *máxima ganancia reducida*: la columna que más mejoraría el LP actual si se agregara. Es un problema de ruteo de un solo vehículo con selección de pacientes.

Sean π_p los duales de (29) y μ_t los de (30) (o de sus cotas en el nodo). El costo reducido de una ruta $r \in \Omega_t$ es

$$\bar{c}_r = c_r - \sum_{p \in P} a_{pr} \pi_p - \mu_t = \sum_{p \in P} a_{pr} (\text{beneficio}[p] - \pi_p) - \text{costo}[t] - \mu_t,$$

por lo que basta maximizar la suma $\sum_p (\text{beneficio}[p] - \pi_p) z_p$ sobre las rutas factibles y restar al final las constantes $\text{costo}[t] + \mu_t$.

ii Conjuntos, Parámetros y Variables de Decisión

Problema Maestro (formulación por rutas)

- P : pacientes; T : tipos de combi
- Ω_t : conjunto (de tamaño exponencial) de rutas factibles para el tipo t ; $\Omega = \bigcup_{t \in T} \Omega_t$
- $a_{pr} \in \{0, 1\}$: parámetro igual a 1 si la ruta r atiende al paciente p

- $c_r = \sum_{p \in P} a_{pr} \cdot \text{beneficio}[p] - \text{costo}[t(r)]$: rentabilidad neta de la ruta r (el beneficio de sus pacientes menos el costo operativo de su combi)
- $\text{disp}[t]$: cantidad de combis disponibles del tipo t
- Variable de decisión: $y_r \in \{0, 1\}$, igual a 1 si la ruta r se ejecuta

Subproblema de Pricing Para un tipo t fijo, con $V = \{0\} \cup P$ los nodos, $n = |P|$, d_{ij} las distancias, $\text{asientos}[t]$ la capacidad y M una constante grande:

- $x_{ij} \in \{0, 1\}$: la combi viaja directamente del nodo i al nodo j
- $z_p \in \{0, 1\}$: la ruta atiende al paciente p
- $T_i \geq 0$: tiempo de llegada al nodo i
- $u_p \in [0, n]$: posición del paciente p en el recorrido (orden tipo MTZ)

iii Formulación Algebraica Completa

Problema Maestro (formulación por rutas)

$$\max \quad Z = \sum_{r \in \Omega} c_r \cdot y_r \quad (28)$$

$$\text{s.a.} \quad \sum_{r \in \Omega} a_{pr} \cdot y_r \leq 1 \quad \forall p \in P \quad (29)$$

$$\sum_{r \in \Omega_t} y_r \leq \text{disp}[t] \quad \forall t \in T \quad (30)$$

$$y_r \in \{0, 1\} \quad \forall r \in \Omega \quad (31)$$

Subproblema de Pricing

$$\max \quad \sum_{p \in P} (\text{beneficio}[p] - \pi_p) \cdot z_p \quad (32)$$

$$\text{s.a.} \quad \sum_{p \in P} z_p \leq \text{asientos}[t] \quad (33)$$

$$\sum_{i \in V, i \neq p} x_{ip} = z_p, \quad \sum_{j \in V, j \neq p} x_{pj} = z_p \quad \forall p \in P \quad (34)$$

$$\sum_{j \in P} x_{0j} \leq 1, \quad \sum_{i \in P} x_{i0} = \sum_{j \in P} x_{0j} \quad (35)$$

$$z_p \leq \sum_{j \in P} x_{0j} \quad \forall p \in P \quad (36)$$

$$T_j \geq T_i + d_{ij} - M \cdot (1 - x_{ij}) \quad \forall i \in V, j \in P, i \neq j \quad (37)$$

$$u_j \geq u_i + 1 - n \cdot (1 - x_{ij}) \quad \forall i, j \in P, i \neq j \quad (38)$$

$$ih_{\text{inicio}}[p] \cdot z_p \leq T_p \leq ih_{\text{fin}}[p] \cdot z_p + M \cdot (1 - z_p) \quad \forall p \in P \quad (39)$$

$$z_p + z_q \leq 1 \quad \forall (p, q) : (\text{categoria}[p], \text{categoria}[q]) \in \mathcal{I} \quad (40)$$

y, según las decisiones del nodo (con J los pares “juntos”, S los pares “separados” y F los pacientes prohibidos):

$$z_a = z_b \quad \forall (a, b) \in J, \quad z_a + z_b \leq 1 \quad \forall (a, b) \in S, \quad z_p = 0 \quad \forall p \in F \quad (41)$$

iv Explicación de la Función Objetivo y Restricciones

Problema Maestro (formulación por rutas) El objetivo (28) suma la rentabilidad neta de las rutas elegidas; coincide con el beneficio neto del enunciado porque cada ruta ya descuenta el costo de su combi. La restricción (29) impide que un paciente sea atendido por más de una ruta (atenderlo o no queda implícito: la desigualdad permite dejarlo sin cubrir), y (30) que se usen más vehículos de un tipo que los disponibles. Como $|\Omega|$ es exponencial, el modelo completo nunca se materializa: en cada nodo del árbol se resuelve la *relajación lineal* ($y_r \geq 0$) sobre el subconjunto de columnas generado hasta el momento —el *maestro restringido*— y el pricing agrega las que falten.

El maestro restringido dentro de un nodo. En un nodo del árbol, el LP se ajusta a las decisiones de ramificación vigentes:

- Participan solo las columnas del pool *compatibles* con el nodo (por ejemplo, en una rama “ p y q separados” se excluyen las rutas que los contienen a ambos).
- Si la ramificación exige atender al paciente p , la restricción (29) pasa a igualdad. Para que el LP nunca se vuelva infactible (y sus duales queden bien definidos) se agrega una variable artificial $s_p \in [0, 1]$ penalizada con un coeficiente ρ en el objetivo: $\sum_r a_{pr} y_r + s_p = 1$. Con ρ mayor que cualquier beneficio alcanzable, $s_p > 0$ solo ocurre si las decisiones del nodo son incompatibles entre sí; en ese caso el nodo se poda.
- Las ramificaciones sobre la flota reemplazan (30) por cotas $lb_t \leq \sum_{r \in \Omega_t} y_r \leq ub_t$ (la cota inferior también lleva su variable artificial penalizada).

Subproblema de Pricing El objetivo (32) maximiza la parte variable de la ganancia reducida: cada paciente “vale” su beneficio menos el precio sombra π_p que el maestro ya paga por cubrirlo. (33) limita los asientos. (34) vincula flujo y atención: la combi entra y sale exactamente una vez de cada paciente atendido y no pasa por los no atendidos. (35) permite a lo sumo un viaje, que si existe sale del centro y regresa a él; (36) impide “rutas fantasma” (atender pacientes sin salir del centro). (37) propaga el tiempo a lo largo de los arcos usados; al ser una desigualdad, la combi puede *esperar* en la puerta hasta ih_{inicio} , tal como exige el enunciado. (39) obliga a recoger a cada paciente atendido dentro de su ventana. (38) es un orden tipo Miller–Tucker–Zemlin entre pacientes: elimina los subtours que (37) no alcanza a prohibir cuando hay pacientes a distancia 0 (dos domicilios en la misma coordenada permitirían un ciclo de duración nula). (40) replica las incompatibilidades de bioseguridad y (41) impone las decisiones de ramificación del nodo. Los pacientes *requeridos* por la ramificación no aparecen aquí: exigir cobertura es una condición global sobre el conjunto de rutas y se impone en el maestro.

Criterio de incorporación. Si θ_t^* es el óptimo de (32), la mejor ruta del tipo t tiene ganancia reducida $\theta_t^* - \text{costo}[t] - \mu_t$. Si supera la tolerancia $\varepsilon = 10^{-4}$, la ruta se agrega al pool. Cuando ningún tipo aporta columnas, el LP del nodo es óptimo y su valor es una cota dual válida para todo el subárbol.

(c) Pseudocódigo y Detalles de Implementación

Los Algoritmos 3.1–3.3 presentan el método en tres niveles: el esquema principal de Branch & Price, la generación de columnas dentro de un nodo (que incluye la eliminación de columnas) y la construcción del pool inicial.

Algoritmo 3.1 SALUDCHALLENGER — esquema principal de Branch & Price

Entrada: instancia (pacientes P , centro, flota, incompatibilidades), umbral de tiempo

Salida: archivo de salida con la mejor solución entera hallada

```
1: leer la instancia y calcular la matriz de distancias;  $fin \leftarrow$  ahora + umbral
2:  $pool \leftarrow$  COLUMNASINICIALES() ▷ Algoritmo 3.3
3:  $(Z^*, R^*) \leftarrow$  MAESTROENTERO( $pool$ ) ▷ warm-start; “no operar” garantiza  $Z^* \geq 0$ 
4:  $abiertos \leftarrow$  {nodo raíz, sin decisiones de ramificación}
5: mientras  $abiertos \neq \emptyset$  y resta tiempo (descontada la reserva final) y nodos < MAX hacer
6:    $n \leftarrow$  nodo de mejor cota heredada de  $abiertos$  ▷ best-first
7:   si  $cota(n) \leq Z^*$  entonces continuar ▷ poda por cota heredada
8:   fin si
9:    $(obj, y, artif, \text{convergió}) \leftarrow$  CGENNODO( $n, pool, fin$ ) ▷ Algoritmo 3.2
10:  si no convergió entonces salir del ciclo ▷ se agotó el tiempo
11:  fin si
12:  si LP infactible o  $artif$  o  $obj \leq Z^*$  entonces continuar ▷ poda
13:  fin si
14:  si  $y$  es entera entonces actualizar  $(Z^*, R^*)$  y continuar
15:  fin si
16:   $d \leftarrow$  primera regla aplicable: Ryan–Foster  $\rightarrow$  atención de paciente  $\rightarrow$  flota por tipo
17:  si no hay regla aplicable entonces ▷ columnas equivalentes
18:    redondear por grupos; actualizar  $(Z^*, R^*)$  si mejora; continuar
19:  fin si
20:  crear los dos hijos de  $n$  según  $d$ , con cota heredada  $obj$ , y agregarlos a  $abiertos$ 
21: fin mientras
22: si  $abiertos = \emptyset$  y no se cortó por tiempo ni por nodos entonces marcar óptimo certificado
23: fin si
24:  $(Z_{ip}, R_{ip}) \leftarrow$  MAESTROENTERO( $pool$ ) con el tiempo restante ▷ refuerzo final
25: si  $Z_{ip} > Z^*$  entonces  $(Z^*, R^*) \leftarrow (Z_{ip}, R_{ip})$ 
26: fin si
27: escribir  $Z^*$ , el itinerario de cada ruta de  $R^*$  y los pacientes no atendidos
```

Algoritmo 3.2 CGENNODO — generación de columnas en un nodo, con eliminación de columnas

Entrada: nodo n , $pool$ global de columnas, fin (instante límite)

Salida: cota LP del nodo, solución y , uso de artificiales, indicador de convergencia

```
1: construir el pricing de cada tipo  $t$  con las restricciones (41) de  $n$  ▷ se reutiliza en todas las iteraciones
2: repetir
3:   si se alcanzó  $fin$  entonces devolver el último LP resuelto, con  $convergió = falso$ 
4:   fin si
5:    $activas \leftarrow \{r \in pool : r \text{ no eliminada y compatible con } n\}$ 
6:   resolver el LP maestro restringido sobre  $activas \Rightarrow obj, y$ , duales  $(\pi, \mu)$ 
7:   para cada  $r \in activas$  hacer ▷ eliminación de columnas sin uso
8:     si  $y_r > 0$  entonces  $sin\_uso_r \leftarrow 0$ 
9:     si no  $sin\_uso_r \leftarrow sin\_uso_r + 1$ 
10:    fin si
11:    si  $sin\_uso_r \geq K$  y  $r$  no pertenece al incumbente entonces marcar  $r$  como eliminada
12:    fin si
13:  fin para
14:   $nuevas \leftarrow 0$ 
15:  para cada tipo de combi  $t$  hacer
16:    resolver el pricing de  $t$  con  $(\pi, \mu_t) \Rightarrow$  mejor ruta  $r^*$  y ganancia reducida  $\bar{c}^*$ 
17:    si  $\bar{c}^* > \varepsilon$  entonces agregar  $r^*$  al pool (si ya existía eliminada, revivirla);  $nuevas \leftarrow nuevas + 1$ 
18:    fin si
19:  fin para
20:  si  $nuevas = 0$  entonces devolver ( $obj, y, artif$ , verdadero) ▷ el LP del nodo es óptimo
21:  fin si
22: fin repetir
```

Algoritmo 3.3 COLUMNAS INICIALES — singletons factibles + heurística golosa

Entrada: pacientes P , centro, flota, distancias d , incompatibilidades

Salida: pool inicial de columnas

```
1:  $p$  es alcanzable  $\iff \max(d_{0p}, ih_{\text{inicio}}[p]) \leq ih_{\text{fin}}[p]$   $\triangleright$  saliendo en  $t = 0$  se llega a tiempo
2: para cada tipo  $t$  y paciente alcanzable  $p$  hacer
3:   agregar la ruta singleton  $[0 \rightarrow p \rightarrow 0]$  del tipo  $t$ 
4: fin para
5: para cada tipo de combi  $t$  hacer  $\triangleright$  rutas multi-paciente golosas
6:    $pend \leftarrow$  pacientes alcanzables, ordenados por beneficio decreciente
7:   mientras  $pend \neq \emptyset$  hacer
8:      $ruta \leftarrow []$ ;  $pos \leftarrow$  centro;  $\tau \leftarrow 0$ 
9:     mientras  $|ruta| < \text{asientos}[t]$  hacer
10:       $c \leftarrow$  el paciente de  $pend$  compatible con  $ruta$  y con ventana cumplible llegando en  $\max(\tau + d_{pos,p}, ih_{\text{inicio}}[p])$  que maximiza  $\text{beneficio}[p] - d_{pos,p}$ 
11:      si no existe tal  $c$  entonces salir del ciclo interno
12:      fin si
13:      agregar  $c$  a  $ruta$ ;  $\tau \leftarrow \max(\tau + d_{pos,c}, ih_{\text{inicio}}[c])$ ;  $pos \leftarrow c$ 
14:    fin mientras
15:    si  $ruta = []$  entonces salir del ciclo  $\triangleright$  ningún paciente pudo iniciar ruta
16:    fin si
17:    si  $|ruta| \geq 2$  entonces agregar la columna  $[0 \rightarrow ruta \rightarrow 0]$  del tipo  $t$   $\triangleright$  los singletons ya están
18:    fin si
19:    quitar los pacientes de  $ruta$  de  $pend$ 
20:  fin mientras
21: fin para
```

Reglas de ramificación.

Por qué no se ramifica sobre y_r . La ramificación clásica sobre una variable ($y_r = 0$ vs. $y_r = 1$) es incompatible con la generación de columnas: en la rama $y_r = 0$ el pricing tendería a regenerar exactamente la ruta prohibida (sigue siendo la más atractiva), y excluir *una ruta puntual* no puede expresarse como restricción lineal del pricing sin romper su estructura. Además el árbol queda desbalanceado: $y_r = 1$ fija mucho y $y_r = 0$ casi nada. Por eso se ramifica sobre cantidades agregadas de la solución, que sí se traducen en restricciones lineales simples del pricing y en filtros sobre el pool.

Si la solución LP del nodo es fraccionaria, se aplica la primera regla utilizable:

1. **Ryan–Foster (pares de pacientes):** se calcula $w_{pq} = \sum_{r: p, q \in r} y_r$ (cuánto “viajan juntos” p y q) y se elige el par de valor más fraccionario (más cercano a 0,5). En la rama *juntos* se exige $z_p = z_q$ en el pricing y se descartan del maestro las columnas que contienen exactamente uno de los dos; en la rama *separados* se exige $z_p + z_q \leq 1$ y se descartan las columnas que los contienen a ambos. Ambas ramas recortan el espacio de forma equilibrada.
2. **Atención de un paciente:** si la cobertura $\alpha_p = \sum_r a_{pr} \cdot y_r$ es fraccionaria, en una rama se fuerza la atención de p (la restricción (29) pasa a igualdad, con su variable artificial penalizada) y en la otra se prohíbe ($z_p = 0$ en el pricing y filtrado de las columnas que lo contienen).
3. **Vehículos por tipo:** si la cantidad de vehículos usados de un tipo, $v_t = \sum_{r \in \Omega_t} y_r$, es fraccionaria, se ramifica en $v_t \leq \lfloor v_t \rfloor$ y $v_t \geq \lceil v_t \rceil$.

Queda un caso borde: columnas *equivalentes* (mismo tipo de combi y mismo conjunto de pacientes, en distinto orden) pueden repartirse valor fraccionario sin que ninguna regla aplique, porque todas las

cantidades agregadas resultan enteras. Como esas columnas tienen idéntica rentabilidad, basta elegir un representante por grupo: se obtiene una solución entera del mismo valor que el LP y el nodo se cierra.

i Detalles de implementación

- **Pool global y deduplicación.** Las columnas viven en un único pool compartido por todos los nodos del árbol: una ruta generada en una rama puede reutilizarse en otra. Cada columna se identifica con la clave (tipo de combi, camino) para no insertar duplicados, y en cada nodo el maestro solo ve las columnas *compatibles* con sus decisiones de ramificación.
- **Eliminación de columnas ($K = 25$).** Cada columna lleva un contador de resoluciones consecutivas del maestro en las que valió $y_r = 0$; al llegar a K se marca como eliminada y deja de entrar en los maestros siguientes. Dos salvaguardas mantienen la validez de las cotas: las columnas de la mejor solución entera conocida están *protegidas* (nunca se eliminan) y, si el pricing vuelve a generar una columna eliminada, ésta se *reactiva* en lugar de descartarse, de modo que nunca se declara la convergencia de un LP con columnas atractivas afuera. Se eligió $K = 25$, más conservador que el valor 5 sugerido en el enunciado: mantener columnas de más solo agranda un LP que ya es chico, mientras que eliminar de más obliga al pricing (un MILP costoso) a regenerarlas.
- **Expiración segura del umbral.** Se fija un instante límite global al inicio, y toda llamada a SCIP (maestros y pricings) recibe `limits/time` acotado por el tiempo restante; el pricing se limita además a 10 segundos por llamada y su plazo se re-verifica antes de cada tipo de combi, de modo que ningún tipo consuma tiempo ya vencido. Antes de explorar se reserva una ventana final de $\min(30, \max(3, 0.05 \cdot \text{umbral}))$ segundos para el maestro entero de refuerzo, y de ella se descuenta un margen para escribir la salida. Si el tiempo se agota en medio de un nodo, se devuelve el último LP resuelto (marcado como no convergido) y se reporta el incumbente vigente.
- **Prioridad de una solución factible.** La combinación de warm-start y reserva final garantiza que la función siempre escriba una solución factible antes de agotar el umbral: aun con umbrales muy cortos existe al menos el incumbente del warm-start y, en el peor caso, la solución trivial “no operar” ($Z = 0$).
- **Obtención de duales en SCIP.** En el maestro LP se desactivan presolve, heurísticas y propagación: si SCIP transforma o reduce el problema, los duales de las restricciones originales se pierden o quedan corruptos. Además, en problemas de maximización SCIP expone los duales de su problema interno de minimización, por lo que deben *negarse* para recuperar los duales verdaderos. Los duales de las cotas superior e inferior de flota de un mismo tipo se suman para formar μ_t .
- **Variables artificiales penalizadas.** La penalización se fija en $\rho = \sum_p |\text{beneficio}[p]| + \sum_t \text{costo}[t] + 1000$, estrictamente mayor que cualquier mejora posible del objetivo: una artificial solo toma valor positivo si las decisiones del nodo son insatisfacibles, y en ese caso el nodo se poda. Su presencia mantiene el LP factible y con duales bien definidos durante toda la generación de columnas.
- **Reutilización de los modelos de pricing.** El MILP de pricing de cada tipo se construye una sola vez por nodo; entre iteraciones de CG solo se actualiza su función objetivo con los duales nuevos (vía `freeTransform` de PySCIPOpt), siguiendo la recomendación del enunciado de modificar dinámicamente un modelo ya ejecutado en lugar de reconstruirlo desde cero.
- **Tolerancias y salvaguardas.** Ganancia reducida mínima $\varepsilon = 10^{-4}$, tolerancia de integralidad 10^{-5} y de poda 10^{-6} ; tope de 500 nodos como resguardo ante árboles patológicos. El certificado de optimalidad solo se declara si el árbol se agotó sin alcanzar ninguno de los límites.

4 Evaluación de las estrategias

(a) Configuración de Evaluación

- **Instancias:** 7 en la carpeta IN/ — cuatro casos de prueba propios (`test1–test4`, de 3 a 7 pacientes) y tres instancias provistas por la cátedra (`instancia1` con 5 pacientes, `instancia8` con 40 y `instancia17` con 100)
- **Umbral de tiempo:** 500 segundos por instancia y modelo
- **Ejecución:** `python evaluador.py config.ini`. El script descubre las instancias en `inPath`, corre cada modelo como subproceso pasándole la instancia, el umbral y las rutas de entrada/salida del `.ini`, y recolecta las métricas de los tags `[METRIC]` de `stdout` y de la línea `Z =` de cada `.out`
- **Métricas capturadas:** las cinco que pide el enunciado (mejor objetivo, cota dual, `#` restricciones, `#` variables y `#` variables en el último maestro)

(b) Tabla Comparativa

La Tabla 1 resume los resultados de las tres estrategias sobre las siete instancias de la carpeta IN/, con un umbral de 500 segundos para cada instancia y modelo. Los valores provienen del `metrics.csv` generado por `evaluador.py`.

Instancia	Métrica	Salud	SaludCG	Challenger
instancia1	Mejor objetivo	620	620	620
	Cota dual	620	640	640
	# restricciones	164	7	7
	# variables	131	21	19
	# variables en últ. maestro	-	23	19
instancia17	Mejor objetivo	2251	18169	17196
	Cota dual	20482.67	19093.70	18844.60
	# restricciones	131968	103	103
	# variables	113422	434	342
	# variables en últ. maestro	-	434	342
instancia8	Mejor objetivo	2610	7010	6735
	Cota dual	7427.27	7238.95	N/A
	# restricciones	21556	43	43
	# variables	18982	189	194
	# variables en últ. maestro	-	231	194
test1	Mejor objetivo	390	390	390
	Cota dual	390	390	390
	# restricciones	178	5	5
	# variables	143	8	10
	# variables en últ. maestro	-	10	10
test2	Mejor objetivo	640	640	640
	Cota dual	640	640	640
	# restricciones	413	8	8
	# variables	341	24	23
	# variables en últ. maestro	-	29	23
test3	Mejor objetivo	1110	1110	1110
	Cota dual	1110	1110	1110
	# restricciones	432	10	10
	# variables	367	30	38
	# variables en últ. maestro	-	42	38
test4	Mejor objetivo	20	20	20
	Cota dual	20	25	25
	# restricciones	76	6	6
	# variables	64	11	20
	# variables en últ. maestro	-	16	20

Table 1: Resultados comparativos de las tres estrategias con un umbral de 500 segundos por instancia y modelo. En cada fila se destaca en **negrita** el mejor valor; cuando las tres estrategias coinciden no se destaca ninguna.

(c) Abreviaciones

- **Mejor objetivo:** Valor de la función objetivo Z encontrado (beneficio neto)
- **Cota dual:** Valor objetivo de la relajación lineal al finalizar la ejecución. Al ser un problema de maximización es una **cota superior** del óptimo entero: cuanto más baja, más ajustada.

Advertencia sobre su validez en SaludCG y Challenger. En Salud el valor lo reporta SCIP y es una cota dual certificada del MILP. En las estrategias con generación de columnas, en cambio, es el valor del LP del maestro *restringido*, que solo constituye una cota superior válida si el pricing demostró que ya no quedan columnas de ganancia reducida positiva. El código da por convergida la generación cuando ningún tipo de combi aporta columnas, sin distinguir si el pricing lo *probó*

(terminó en **optimal**) o simplemente agotó su límite de 10 segundos. En **instancia17** ocurre lo segundo: los subproblemas de los tres tipos de combi se cortan por tiempo, de modo que los valores 19093,70 y 18844,60 deben leerse como valores de referencia y no como cotas certificadas. En esa fila la única cota rigurosa es la de Salud. Véase la Sección 5

- **# variables:** Cantidad de variables del **modelo inicial**. En Salud son las del MILP compacto antes del *presolve* (x , z , a , u y T); en SaludCG y Challenger, las columnas del pool inicial con que arranca el maestro
- **# restricciones:** Cantidad de restricciones del **modelo inicial**. En SaludCG y Challenger son las del maestro ($|P| + |T|$: una de cobertura por paciente y una de disponibilidad por tipo de combi)
- **# variables en últ. maestro:** Cantidad de columnas del Modelo Maestro en el momento en que se detiene la generación de columnas, es decir antes de resolver el maestro entero final. Solo aplica a las estrategias con generación de columnas
- -: No aplicable (Salud no usa generación de columnas)
- **N/A:** No hay cota dual disponible. En Challenger sobre **instancia8** significa que la generación de columnas del nodo raíz no llegó a converger dentro del umbral (7 iteraciones, con el pricing todavía produciendo columnas), por lo que la estrategia se abstiene de reportar un valor que no sería una cota. Es, paradójicamente, el dato más honesto de esa fila

5 Conclusiones

(a) Hallazgos Principales

- **Las tres estrategias coinciden en las instancias chicas.** En `test1–test4` e `instancia1` (hasta 7 pacientes) los tres métodos devuelven el mismo objetivo, y Salud lo certifica como óptimo: su cota dual coincide con el objetivo entero. En `test1–test3` la relajación por rutas ya resulta entera en el nodo raíz de Challenger; en `test4` el LP raíz es fraccionario (cota 25.0 frente al óptimo entero 20.0) y hace falta ramificar para cerrar el gap. Nótese que en `test4` e `instancia1` la cota más ajustada es la del modelo compacto, no la de las descomposiciones.
- **En las instancias grandes el modelo compacto se derrumba.** Éste es el resultado central de la evaluación. Con el mismo umbral de 500 segundos, en `instancia8` (40 pacientes) Salud llega a $Z = 2610$ contra 7010 de SaludCG, y en `instancia17` (100 pacientes) a $Z = 2251$ contra 18169: un factor de ocho. La causa es el tamaño del modelo: `instancia17` genera 113 422 variables y 131 968 restricciones, y SCIP consume el presupuesto entero sin cerrar el gap (su cota dual queda en 20 482, casi diez veces el mejor entero que encontró). El maestro de las descomposiciones, en cambio, trabaja con 103 restricciones y unos cientos de columnas.
- **Eficiencia estructural del maestro.** La formulación por rutas traslada la complejidad del ruteo al subproblema de pricing y deja un maestro diminuto: en `instancia17`, 103 restricciones frente a 131 968, y 342–434 columnas frente a 113 422 variables. Esa compacidad es la que permite seguir produciendo soluciones de calidad cuando el compacto ya no puede.
- **El costo lo paga el pricing.** La contracara es que cada iteración de generación de columnas exige resolver un MILP de ruteo por tipo de combi. En las instancias chicas todo termina en menos de un segundo, pero en `instancia8` SaludCG consume 416 s y Challenger 477 s, y en `instancia17` Challenger vuelve a agotar su presupuesto (476 s) sin certificar optimalidad. En ambos casos Challenger se detiene por el límite de tiempo, reservando los segundos finales para el maestro entero de refuerzo.
- **Garantía de Factibilidad:** El módulo de validación independiente (`SaludTest`) operó de forma correcta en todas las corridas, permitiendo verificar de manera estricta y sin recurrir a programación lineal que las soluciones entregadas cumplen a cabalidad con las capacidades de la flota, las ventanas horarias y las reglas de bioseguridad.
- **¿Cómo se prioriza encontrar una solución factible antes de agotar el tiempo?** Cada estrategia implementa mecanismos distintos de garantía de factibilidad. Por ejemplo, en la estrategia 1 al agotarse el tiempo, se retorna la mejor solución entera encontrada hasta ese momento (aunque no sea óptima). Mientras que en la estrategia 2, se utiliza el último Maestro Entero para extraer una solución factible. Por último, la estrategia 3 retorna la mejor solución entera encontrada en cualquier nodo del árbol.

(b) Comportamientos Sorprendentes

- **Degeneración Dual y Sensibilidad del Solver:** Durante el desarrollo de la Estrategia 2 (SaludCG), se constató un comportamiento crítico derivado del motor de resolución SCIP: el mecanismo interno de *presolve* tendía a eliminar restricciones con una sola variable (convirtiéndolas en simples cotas de dominio), lo que anulaba la recuperación de los precios sombra (π_p y μ_k). Esto generó inicialmente un estancamiento en el bucle de pricing que se resolvió exitosamente forzando restricciones de tipo `modifiable=True` y aplicando la inversión de signo correspondiente a la maximización.
- **En `instancia17`, la generación de columnas no generó ninguna columna.** El resultado más inesperado de la evaluación. SaludCG terminó en 32 s de los 500 disponibles, y el maestro final

tiene exactamente las mismas 434 columnas que el pool inicial. Es decir: hizo una única iteración, el pricing — acotado a 10 segundos por llamada, sobre un MILP de ruteo de 100 pacientes — no logró exhibir ninguna ruta de ganancia reducida positiva, y el bucle cortó por falta de columnas nuevas. *Todo* el $Z = 18169$ que reporta proviene de la heurística golosa de inicialización, no del método de generación de columnas. Esto invierte la lectura ingenua de la tabla: SaludCG no gana por descomponer mejor, sino porque su pool inicial ya es una buena solución mientras que el compacto ni siquiera consigue una razonable.

- **La calidad del pool inicial pesa más que el refinamiento.** Como corolario del punto anterior, en instancias grandes la heurística golosa resultó ser el componente decisivo y el pricing exacto, el cuello de botella. Challenger, que invierte sus 500 segundos en explorar el árbol y refinar columnas, termina *por debajo* de SaludCG en las dos instancias grandes (17 196 contra 18 169 y 6735 contra 7010): el esfuerzo adicional se consume en pricings caros en lugar de en diversificar el pool.
- **El pricing puede *simular* una convergencia que no ocurrió.** El hallazgo más sutil, y el que más nos hizo desconfiar de la lectura directa de la tabla. Comparando las dos instancias grandes aparece una anomalía: Challenger reporta cota dual en *instancia17* (100 pacientes) pero N/A en *instancia8* (40), es decir que *converge en la más grande y no en la más chica*. Al medir los subproblemas con el límite de 10 segundos por llamada que usa la implementación, la causa quedó clara:

Instancia	Combi_Chica	Combi_Mediana	Combi_Grande
instancia8	optimal (2,2 s)	optimal (6,9 s)	timelimit
instancia17	timelimit	timelimit	timelimit

En *instancia17* los tres subproblemas se cortan por tiempo sin resolver. Como ninguno devuelve una columna, el bucle interpreta “no quedan columnas que mejoren” y da la generación por convergida — pero eso no es una demostración, es un vencimiento de plazo. En *instancia8*, en cambio, dos de los tres pricings sí resuelven a optimalidad, la generación sigue produciendo columnas de verdad y se queda sin tiempo antes de terminar: por eso reporta N/A.

La conclusión invierte la apariencia de la tabla. El N/A de *instancia8* no es una carencia sino el dato honesto: la estrategia se abstiene de publicar un número que no sería una cota. Y el 18 844,60 de *instancia17*, que a primera vista luce como la cota más ajustada de su fila, es el valor del LP de un maestro restringido cuya optimalidad nadie certificó. En esa fila la única cota rigurosa es la de Salud (20 482,67), que proviene del solver como cota dual del MILP completo. La lección de método: en generación de columnas, “el pricing no encontró nada” y “el pricing probó que no hay nada” son afirmaciones muy distintas, y conviene que el código las distinga antes de publicar una cota.

- **Cotas duales muy flojas en el compacto.** En *instancia17* la cota dual de Salud (20 482) es casi diez veces su mejor solución entera (2251). El Big-M de las ventanas de tiempo relaja tanto la formulación que la relajación lineal deja de ser informativa, y el *gap* reportado por el solver no permite distinguir una solución mediocre de una buena.

(c) Dificultades y Conocimientos Adquiridos

- **Linealización y Acotamiento lógico:** El modelado de las ventanas de tiempo y la eliminación de subtours exigió un uso intensivo de constantes Big-M. Se aprendió la importancia crítica de dimensionar correctamente estas cotas para evitar espacios factibles artificialmente relajados o problemas numéricos de mal condicionamiento en el solver.
- **Gestión de Duales en Descomposiciones:** Comprender la interacción matemática entre los precios sombra del Problema Maestro y la función objetivo del Subproblema de Pricing (maximización de la ganancia reducida) afianzó los conceptos teóricos de la programación entera y la

dualidad de Lagrange aplicadas a la logística.

- **Validación Independiente:** La implementación de rutinas de verificación desacopladas del entorno de optimización lineal (**SaludTest**) reforzó las buenas prácticas de ingeniería de software, permitiendo aislar errores de ruteo y garantizar la trazabilidad estricta de las salidas institucionales.
- **PYSCIP:** Uso de la API de Python para SCIP en problemas reales, incluyendo la manipulación de modelos dinámicos, la actualización de objetivos y la recuperación de duales.
- **LaTeX:** Uso de LaTeX para la documentación y presentación de resultados, permitiendo generar reportes profesionales y correctamente estructurados.

6 Apéndice: módulo de validación SaludTest

Además de las tres estrategias, el enunciado exige una función auxiliar de validación `SaludTest(instancia, output)`: una función *booleana* que lee un archivo de salida ya generado y decide si la solución que describe es *operativamente factible* contrastándola contra los datos de la instancia. Es común a las tres estrategias — valida cualquier `.out` sin importar qué modelo lo produjo — y por eso se describe aparte.

La restricción central es que **no puede usar programación lineal**. La verificación es puramente aritmética: se reconstruye el itinerario de cada combi paso a paso, se acumula el tiempo de viaje y se contrasta cada condición del enunciado. Ningún solver interviene, de modo que la validación es independiente del modelo que generó la solución y no puede “heredar” un error de éste.

Qué verifica

1. **Formación del itinerario.** Cada ruta debe empezar y terminar en el centro médico ($\text{id} = 0$), y el tipo de combi declarado debe existir en la flota.
2. **Capacidad.** La cantidad de pacientes de cada ruta no puede superar los asientos de su tipo de combi. Como cada paciente ocupa exactamente un asiento y nadie desciende antes del centro, basta contar los nodos intermedios.
3. **Ventanas de tiempo y tiempos de traslado.** Partiendo de $t = 0$ en el centro, se acumula la distancia euclídea entre nodos consecutivos (velocidad 1, es decir tiempo = distancia). Si la combi llega antes de ih_{inicio} *espera* — el tiempo se adelanta a ih_{inicio} , tal como permite el enunciado —, y si llega después de ih_{fin} la solución se rechaza.
4. **Incompatibilidades de bioseguridad.** Para cada par de pacientes que comparten combi se verifica que sus categorías no pertenezcan al conjunto $\mathcal{I}$.

A esos cuatro puntos que pide el enunciado se agregaron tres verificaciones más, sin las cuales una solución infactible podía pasar como válida:

5. **Disponibilidad de la flota.** La cantidad de rutas de un tipo no puede superar su `cant_disponible`. Sin este control, una solución que usara tres combis de un tipo que solo tiene dos unidades era aceptada.
6. **Unicidad de atención.** Ningún paciente puede aparecer en más de una ruta.
7. **Consistencia del reporte.** El valor Z declarado debe coincidir con $\sum$ beneficio de los atendidos — $\sum$ costo de las combis usadas, y la lista `No_Atendidos` debe ser exactamente el complemento de los atendidos.

Pseudocódigo

Algoritmo 6.1 SALUDTEST — validación de factibilidad sin programación lineal

Entrada: instancia (pacientes P , centro, flota, incompatibilidades), archivo *output*

Salida: **verdadero** si la solución es operativamente factible; **falso** si no

```

1: leer la instancia; parsear del output:  $Z$ , la lista de rutas y No_Atendidos
2:  $usos \leftarrow$  diccionario vacío;  $atendidos \leftarrow \emptyset$ 
3: para cada ruta ( $tipo, [n_0, \dots, n_L]$ ) del output hacer
4:   si  $tipo \notin \text{flota}$  o  $n_0 \neq 0$  o  $n_L \neq 0$  entonces devolver falso           ▷ (1) itinerario mal formado
5:   fin si
6:    $usos[tipo] \leftarrow usos[tipo] + 1$ ;  $S \leftarrow [n_1, \dots, n_{L-1}]$ 
7:   si  $|S| > \text{asientos}[tipo]$  entonces devolver falso                       ▷ (2) capacidad
8:   fin si
9:    $\tau \leftarrow 0$ ;  $pos \leftarrow \text{centro}$ 
10:  para cada  $p \in S$  en orden hacer
11:     $\tau \leftarrow \tau + d(pos, p)$                                            ▷ velocidad 1: tiempo = distancia
12:     $\tau \leftarrow \max(\tau, ih_{\text{inicio}}[p])$                                ▷ si llega antes, espera
13:    si  $\tau > ih_{\text{fin}}[p]$  entonces devolver falso                           ▷ (3) ventana violada
14:    fin si
15:     $pos \leftarrow p$ 
16:  fin para
17:  para cada par  $\{p, q\} \subseteq S$  hacer
18:    si  $(\text{categoria}[p], \text{categoria}[q]) \in \mathcal{I}$  entonces devolver falso       ▷ (4) incompatibilidad
19:    fin si
20:  fin para
21:  si  $S \cap \text{atendidos} \neq \emptyset$  entonces devolver falso                 ▷ (6) paciente repetido
22:  fin si
23:   $\text{atendidos} \leftarrow \text{atendidos} \cup S$ 
24: fin para
25: para cada tipo  $t$  en  $usos$  hacer
26:   si  $usos[t] > \text{cant\_disponible}[t]$  entonces devolver falso             ▷ (5) flota
27:   fin si
28: fin para
29:  $Z^{\text{calc}} \leftarrow \sum_{p \in \text{atendidos}} \text{beneficio}[p] - \sum_{(tipo, \cdot)} \text{costo}[tipo]$ 
30: si  $|Z - Z^{\text{calc}}| > \varepsilon$  o  $\text{No\_Atendidos} \neq P \setminus \text{atendidos}$  entonces devolver falso   ▷ (7) reporte
    inconsistente
31: fin si
32: devolver verdadero

```

Nótese que el recorrido temporal de la línea 9 es también la verificación de que “el recorrido esté bien calculado” que pide el enunciado: si el orden de visita declarado en el *.out* no fuera consistente con las distancias y las ventanas, el acumulador τ lo detecta. Las 21 salidas de la entrega (7 instancias $\times$ 3 estrategias) pasan esta validación.